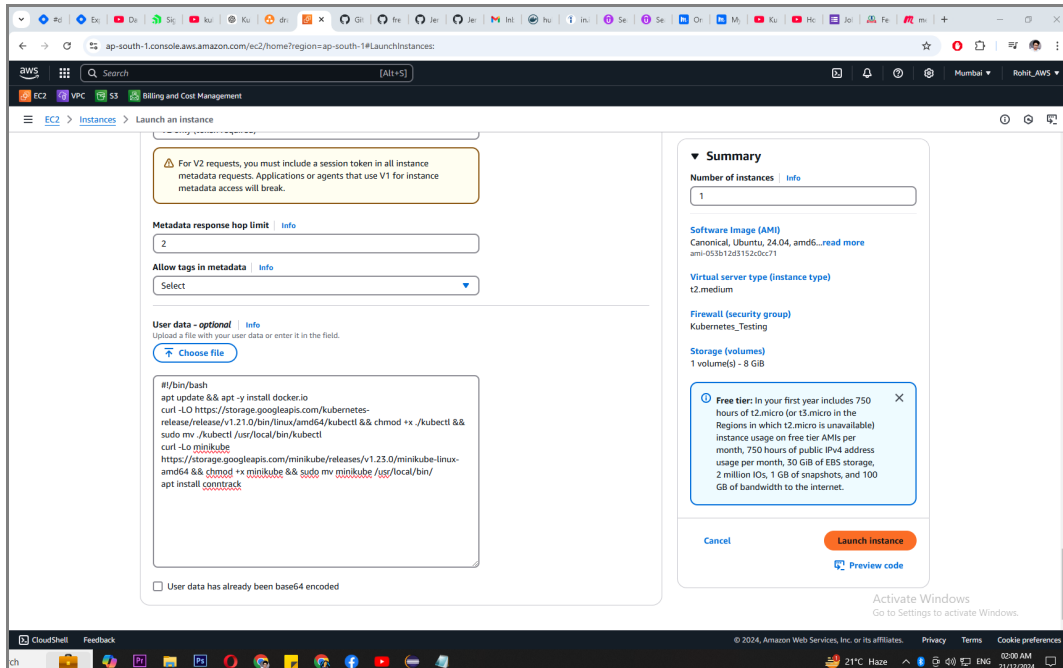

ConfigMap in Kubernetes

Rohit Surwade

Start an EC2 instance with following bash script for installing Kubernetes tools and dependencies.



1.

Script:

```
#!/bin/bash
apt update && apt -y install docker.io
curl -LO https://storage.googleapis.com/kubernetes-release/release/v1.21.0/bin/linux/amd64/kubectl && chmod +x ./kubectl &&
sudo mv ./kubectl /usr/local/bin/kubectl
curl -Lo minikube
https://storage.googleapis.com/minikube/releases/v1.23.0/minikube-linux-amd64 && chmod +x minikube && sudo mv minikube /usr/local/bin/
apt install contrack
```

Start the Minikube Cluster.

```
root@ip-172-31-36-4:/home/ubuntu# minikube start --vm-driver=none
* minikube v1.23.0 on Ubuntu 24.04 (xen/amd64)
* Using the none driver based on user configuration
* Starting control plane node minikube in cluster minikube
* Running on localhost (CPUs=2, Memory=3912MB, Disk=6869MB) ...
* OS release is Ubuntu 24.04.1 LTS
* minikube 1.34.0 is available! Download it: https://github.com/kubernetes/minikube/releases/tag/v1.34.0
* To disable this notice, run: 'minikube config set WantUpdateNotification false'

* Preparing Kubernetes v1.22.1 on Docker 26.1.3 ...
- kubelet.resolv-conf=/run/systemd/resolve/resolv.conf
  > kubelet.sha256: 64 B / 64 B [-----] 100.00% ? p/s 0s
  > kubect1.sha256: 64 B / 64 B [-----] 100.00% ? p/s 0s
  > kubeadm.sha256: 64 B / 64 B [-----] 100.00% ? p/s 0s
  > kubect1: 44.73 MiB / 44.73 MiB [-----] 100.00% 10.45 MiB p/s 4.5s
  > kubeadm: 43.71 MiB / 43.71 MiB [-----] 100.00% 7.72 MiB p/s 5.9s
  > kubelet: 146.25 MiB / 146.25 MiB [-----] 100.00% 14.12 MiB p/s 11s
- Generating certificates and keys ...
```

2.

commands:

minikube start --vm-driver=none


```
root@ip-172-31-36-4:/home/ubuntu# A sample.conf
root@ip-172-31-36-4:/home/ubuntu# B kubect1 create configmap mymap --from-file=sample.conf
configmap/mymap created
root@ip-172-31-36-4:/home/ubuntu# kubect1 get configmap
NAME          DATA      AGE
kube-root-ca.crt 1          3m
mymap          1          10s
root@ip-172-31-36-4:/home/ubuntu#
```

4.

commands:

1. `kubect1 create configmap mymap --from-file=sample.conf`

(This command creates a ConfigMap named mymap in Kubernetes, using the contents of the file sample.conf.

--from-file=sample.conf: Specifies the file whose contents will be added to the ConfigMap.

The ConfigMap will contain one key-value pair:

Key: The filename (sample.conf).

Value: The contents of the file sample.conf.)

2. `kubect1 get configmap`

(To list the configmap objects)

Two ways to verify the ConfigMap.

```

root@ip-172-31-36-4: /home/ubuntu
kube-root-ca.crt 1 3m
mymap 1 10s
root@ip-172-31-36-4: /home/ubuntu# A kubectl get configmap mymap -o yaml
apiVersion: v1
data:
  sample.conf: |
    APP_ENV=production
    APP_PORT=8080
    LOG_LEVEL=info
    DATABASE_URL=postgres://user:password@db.example.com:5432/appdb
kind: ConfigMap
metadata:
  creationTimestamp: "2024-12-21T07:28:54Z"
  name: mymap
  namespace: default
  resourceVersion: "574"
  uid: ad8d0623-3edb-437c-b1b8-89164174e176
root@ip-172-31-36-4: /home/ubuntu# B kubectl describe configmap mymap
Name:         mymap
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
sample.conf:
-----
APP_ENV=production
APP_PORT=8080
LOG_LEVEL=info
DATABASE_URL=postgres://user:password@db.example.com:5432/appdb

Events:  <none>
root@ip-172-31-36-4: /home/ubuntu#

```

5.

commands:

1. `kubectl get configmap <configmap-name> -o yaml`
2. `kubectl describe configmap <configmap-name>`

Manifest to consume configmap(mounted as a volume)

```

apiVersion: v1
kind: Pod
metadata:
  name: myvolconfig
spec:
  containers:
  - name: c1
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo hello-rohit; sleep 5; done"]
    volumeMounts:
    - name: testconfigmap
      mountPath: "/tmp/config" # Config files will be mounted here
  volumes:
  - name: testconfigmap
    configMap:
      name: mymap # Ensure this matches an existing ConfigMap name
      items:
      - key: sample.conf # File will be mounted as /tmp/config/sample.conf
        path: sample.conf

```

-- INSERT --

20,1 All

6.

(This Kubernetes manifest defines a Pod named myvolconfig with a single container c1 running the centos image. The container continuously prints "hello-rohit" every 5 seconds. A volume named testconfigmap is mounted to /tmp/config inside the container, backed by the ConfigMap mymap. The items field ensures that only the key sample.conf from the ConfigMap is mounted as a file named sample.conf at /tmp/config/sample.conf. The type of ConfigMap consumption here is mounted as a volume, ideal for applications requiring configuration data as files. Ensure the ConfigMap mymap exists with the sample.conf key for this setup to work. After deployment, the container can access the file at /tmp/config/sample.conf with the ConfigMap's content.)

```
apiVersion: v1
kind: Pod
metadata:
  name: myvolconfig
spec:
  containers:
  - name: c1
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo hello-rohit; sleep 5; done"]
    volumeMounts:
    - name: testconfigmap
      mountPath: "/tmp/config" # Config files will be mounted here
  volumes:
  - name: testconfigmap
    configMap:
      name: mymap # Ensure this matches an existing ConfigMap name
      items:
      - key: sample.conf
        path: sample.conf # File will be mounted as /tmp/config/sample.conf
```

Applying the Manifest(our pod is created successfully)

```

root@ip-172-31-36-4:/home/ubuntu# A deployconfigmap.yml
root@ip-172-31-36-4:/home/ubuntu# B kubectl apply -f deployconfigmap.yml
pod/myvolconfig created
root@ip-172-31-36-4:/home/ubuntu# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
myvolconfig   1/1     Running   0           16s
root@ip-172-31-36-4:/home/ubuntu#

```

7.

commands:

1. kubectl apply -f <yaml-filename>
2. kubectl get pods

Enter the container and verify if the ConfigMap is reflecting.

```

root@ip-172-31-36-4:/home/ubuntu# vi deployconfigmap.yml
root@ip-172-31-36-4:/home/ubuntu# kubectl apply -f deployconfigmap.yml
pod/myvolconfig created
root@ip-172-31-36-4:/home/ubuntu# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
myvolconfig   1/1     Running   0           16s
root@ip-172-31-36-4:/home/ubuntu# A kubectl exec myvolconfig -it -- /bin/bash
[root@myvolconfig /]# cd /tmp/config
[root@myvolconfig config]# D sample.conf
[root@myvolconfig config]# E cat sample.conf
APP_ENV=production
APP_PORT=8080
LOG_LEVEL=info
DATABASE_URL=postgres://user:password@db.example.com:5432/appdb
[root@myvolconfig config]#

```

8.

IT IS REFLECTING!!!

Manifest to consume configmap(as an environment variable)

```
root@ip-172-31-36-4:~# cat myenvconfig.yaml
apiVersion: v1
kind: Pod
metadata:
  name: myenvconfig
spec:
  containers:
  - name: cl
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo Hello-Rohit; sleep 5; done"]
    env:
    - name: MYENV          # Environment variable to store the value of the ConfigMap key
      valueFrom:
        configMapKeyRef:
          name: mymap      # Name of the ConfigMap containing the key
          key: sample.conf # Key in the ConfigMap whose value is stored in the environment variable
```

9.

(This YAML defines a Kubernetes Pod named myenvconfig with a single container c1 running the centos image. The container continuously echoes "Hello-Rohit" every 5 seconds using a command executed in /bin/bash. The ConfigMap is consumed as an environment variable, where the value of a specific key (sample.conf) from the ConfigMap mymap is injected into the container's environment variable MYENV.

This type of ConfigMap consumption is particularly useful when application processes need to access configuration values through environment variables rather than files. For example, if the sample.conf key in the ConfigMap contains the value APP_ENV=production, the container will have an environment variable MYENV with this value. This approach ensures that configuration data can be dynamically managed without altering the container image,

providing flexibility and modularity in deployment setups. Always ensure the referenced ConfigMap and its key exist before deploying this pod to avoid runtime errors. You can verify the environment variable inside the container by using echo \$MYENV.)

```
apiVersion: v1
kind: Pod
metadata:
  name: myenvconfig
spec:
  containers:
  - name: c1
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo Hello-Rohit; sleep 5; done"]
    env:
    - name: MYENV # Environment variable to store the value of the ConfigMap
      key
      valueFrom:
        configMapKeyRef:
          name: mymap # Name of the ConfigMap containing the key
          key: sample.conf # Key in the ConfigMap whose value is stored in the
            environment variable
```

Applying the Manifest(our pod is created successfully)

```
root@ip-172-31-36-4:/home/ubuntu# kubectl delete -f deployconfigmap.yml
pod "myvolconfig" deleted
root@ip-172-31-36-4:/home/ubuntu# vi deployenv.yml
root@ip-172-31-36-4:/home/ubuntu# kubectl apply -f deployenv.yml
pod/myenvconfig created
root@ip-172-31-36-4:/home/ubuntu# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
myenvconfig   1/1     Running   0           7s
root@ip-172-31-36-4:/home/ubuntu#
```

10.

Enter the container and verify if the ConfigMap is reflecting (using echo \$MYENV).

```
root@ip-172-31-36-4:/home/ubuntu# kubectl apply -f deployenv.yml
pod/myenvconfig created
root@ip-172-31-36-4:/home/ubuntu# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
myenvconfig   1/1     Running   0           7s
root@ip-172-31-36-4:/home/ubuntu# kubectl exec -it myenvconfig -- /bin/bash
[root@myenvconfig /]# echo $MYENV
APP_ENV=production APP_PORT=8080 LOG_LEVEL=info DATABASE_URL=postgres://user:password@db.example.com:5432/appdb
[root@myenvconfig /]#
```

11.

IT IS REFLECTING!!!

We can also check it using (env command)

```
@myenvconfig/
root@ip-172-31-36-4:/home/ubuntu# kubectl apply -f deployenv.yml
pod/myenvconfig created
root@ip-172-31-36-4:/home/ubuntu# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
myenvconfig   1/1     Running   0           7s
root@ip-172-31-36-4:/home/ubuntu# kubectl exec -it myenvconfig -- /bin/bash
[root@myenvconfig /]# echo $MYENV
APP_ENV=production APP_PORT=8080 LOG_LEVEL=info DATABASE_URL=postgres://user:password@db.example.com:5432/appdb
[root@myenvconfig /]# env
LANG=en_US.UTF-8
BKTNAME=myenvconfig
MYENV=APP_ENV=production
APP_PORT=8080
LOG_LEVEL=info
DATABASE_URL=postgres://user:password@db.example.com:5432/appdb
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
PWD=/
HOME=/root
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
TERM=xterm
SHLVL=1
KUBERNETES_SERVICE_PORT=443
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
KUBERNETES_SERVICE_HOST=10.96.0.1
LESSOPEN=||/usr/bin/lesspipe.sh %s
_/usr/bin/env
[root@myenvconfig /]#
```

12.